

DESIGN AND DEVELOP A REACT JS BASED WEB APPLICATION FOR TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

POKAPUDI THARUN KUMAR (VTU24968)

10211CS224

Full Stack Application Development

Slot : E2



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "DESIGN AND DEVELOP A REACT JS BASED WEB APPLICATION FOR TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT" by POKAPUDI THARUN KUMAR (VTU24968) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.M.MAGESHWARI.

Associate Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Co-ordinator of the Department

Dr. M.Sumithra

Assistent Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(POKAPUDI THARUN KUMAR)

Date:06/05/2026

ABSTRACT

Event management and ticket booking for internal departmental activities are often handled manually, leading to inefficiencies and lack of transparency. This project presents **EventFlow**, a full-stack web-based ticket booking system developed using **React JS**, **Spring Boot**, and **PostgreSQL** to streamline event management. The system enables users to register, verify accounts through OTP-based email authentication, browse events, and book tickets securely with **JWT-based authentication** and role-based access control. Key features include real-time ticket availability tracking, a wallet-based payment system, automated email notifications, and an admin dashboard for monitoring and management. By leveraging PostgreSQL for reliable data storage and transaction handling, the system improves operational efficiency and provides a scalable solution for managing internal departmental events.

Keywords: Ticket Booking System, SMTP Email Service, OTP Verification, JWT Authentication, PostgreSQL

LIST OF FIGURES

1.1	Modern web architecture	2
1.2	Booking techniques	4
1.3	Booking techniques	5
1.4	Authentication and security	9
2.1	Event manage tool	14
3.1	Frame work	17
4.1	data layer	25
4.2	DFD Level 0	27
4.3	DFD Level 1	28
5.1	login page	31
5.2	Verification page	32
5.3	Module page	32
5.4	Booking page	33
5.5	Wallet management	33

LIST OF TABLES

3.1	Consolidated System Technology Stack	23
-----	--	----

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CI	Continuous Integration
CSS	Cascading Style Sheets
DBMS	Database Management System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
ORM	Object Relational Mapping
REST	Representational State Transfer

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	3
1.3 Scope of the Project	4
1.4 Framework	6
1.4.1 1.4.1 Backend Framework: Spring Boot	6
1.4.2 1.4.2 Frontend Framework: React.js	7
1.4.3 1.4.3 Database and Persistence: PostgreSQL .	7
1.4.4 1.4.4 Authentication and Security	8

1.4.5	1.4.5 Communication and Services	8
2	SURVEY OF SIMILAR APPLICATION IN MARKET	10
2.1	Survey of Similar Applications in Market	10
2.1.1	Eventbrite	10
2.1.2	Zoho Backstage	11
2.1.3	Ticket Tailor	11
2.1.4	Cvent	11
2.1.5	TicketSource	12
2.1.6	Analysis of Existing Systems	12
2.1.7	Need for Proposed System	13
3	FRAMEWORK DESCRIPTION	15
3.1	Environment Setup	15
3.1.1	Structure of the Project	15
3.1.2	Execution Procedure	16
3.2	Backend Implementation	17
3.2.1	Environment Setup	17
3.2.2	Structure of the Project	18
3.2.3	Execution Procedure	19
3.3	Database Implementation	20
3.3.1	Database Configuration	21
3.3.2	Schema Structure	21

3.3.3	Tables Created	22
3.4	System Technology Stack Overview	22
4	METHODOLOGIES	24
4.1	Project Architecture	24
4.2	Data Flow Diagram (DFD)	26
4.2.1	DFD Level 0: Context Diagram	26
4.2.2	DFD Level 1: Functional Decomposition	26
4.3	Advantages of this Project	27
5	RESULTS AND DISCUSSIONS	29
5.1	System Implementation Overview	29
5.2	Module Explanations	31
5.2.1	User Authentication – Login Page	31
5.2.2	User Verification Module	31
5.2.3	Events Module	32
5.2.4	Ticket Booking Module	32
5.2.5	Wallet Management System	33
6	CONCLUSION AND FUTURE ENHANCEMENTS	34
6.1	Conclusion	34
6.2	Future Enhancements	35
7	References	37

Chapter 1

INTRODUCTION

1.1 Introduction

In recent years, the management of internal departmental events such as technical fests, seminars, and workshops has become an essential part of academic institutions. However, in many cases, the process of event registration and ticket booking is still handled manually or through disconnected systems, leading to inefficiencies, lack of transparency, and increased administrative workload. These traditional approaches often result in errors in ticket allocation, delays in communication, and poor user experience for both organizers and participants.

With the advancement of web technologies, there is a growing need for a centralized and automated system that can efficiently manage event booking and user interactions. A digital platform can simplify the entire process by providing real-time access to event details,



Figure 1.1: Modern web architecture

figure[1.1]:shows enabling secure registration, and ensuring smooth ticket booking operations. Additionally, incorporating secure authentication mechanisms and reliable data management systems can significantly enhance the overall system performance and user trust.

This project presents EventFlow, a full-stack web-based ticket booking system designed for internal departmental events. The system is developed using React JS for the frontend, Spring Boot for the backend, and PostgreSQL as the database for efficient and reliable data storage. It provides features such as user registration with OTP-based email verification using SMTP, secure login through JWT authentication, real-time ticket availability tracking, and a wallet-based payment system. The platform also includes an administrative interface for managing events, monitoring bookings, and ensuring efficient system . Modern web architecture

shows the system architecture of the Smart Event Ticket Booking System.The architecture follows a layered design for better scalability and maintainability.It consists of the user layer, frontend layer, back-

end layer, and database layer. The user layer includes interacting with the system.

1.2 Aim of the Project

- **To develop an automated pest control system**

The aim is to design a smart system that can automatically attract and trap pests using UV light and CO₂ without requiring continuous human involvement, making pest control more efficient and reliable.

- **To enable real-time pest detection and response**

The system uses sensors to detect pest movement and instantly activates the trapping mechanism, ensuring quick action and reducing crop damage caused by pests.

- **To reduce dependency on chemical pesticides**

By using a physical trapping method instead of harmful chemicals, the project promotes an eco-friendly approach that helps maintain soil quality and protects human health.

- **To provide data collection and analysis**

The system records information about the number and frequency of insects captured, allowing farmers to analyze pest activity and make better agricultural decisions.

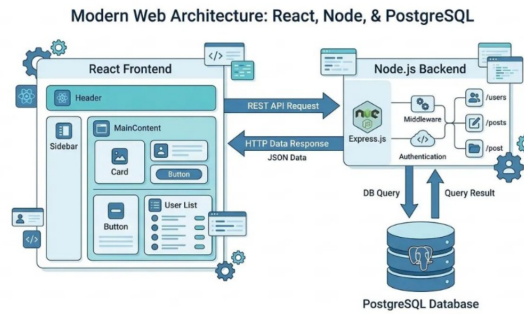


Figure 1.2: Booking techniques

- **To ensure easy maintenance and cost-effectiveness**

The project aims to create a low-cost and durable system that provides maintenance alerts, making it practical for long-term use in agricultural environments. figure[1.2]:The image illustrates a modern web architecture that integrates React for the frontend, Node.js with Express.js for the backend, and PostgreSQL for data storage. In this structure, the React frontend is responsible for building an interactive user interface using components such as headers, sidebars, and content sections. It communicates with the backend through REST API requests, sending and receiving data in JSON format.

1.3 Scope of the Project

- **Application in agricultural fields**

The system can be implemented in farms, gardens, and green-houses to monitor pest activity and control infestations, thereby

improving crop protection and overall productivity.

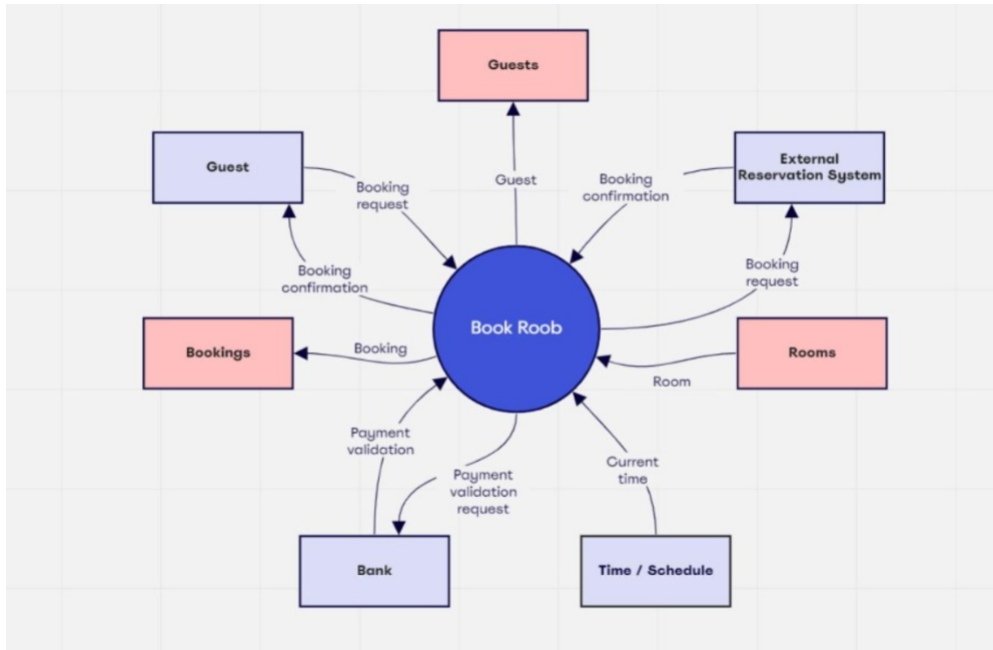


Figure 1.3: Booking techniques

Integration with smart technologies

figure[1.4]:The figure shows the project can be extended with IoT-based platforms for remote monitoring and control, allowing users to access real-time data and system status through mobile or web applications.

Scalability and flexibility

figure[1.3]:The figure shows system can be adapted for different scales of agricultural use, from small farms to large fields, and can be modified to target various types of pests by adjusting sensors and attractants.

Data-driven insights

The collected data can be used to analyze pest patterns, helping farm-

ers predict outbreaks and make informed decisions for better crop management.

Future enhancements

The scope includes further improvements such as AI-based pest identification, solar-powered operation, and increased automation to enhance efficiency and sustainability.

1.4 Framework

figure[1.4]:The development of the proposed Ticket Booking System follows a multi-tier architecture that integrates frontend, backend, and database components to ensure scalability, security, and efficient performance. Each layer is designed to handle specific responsibilities, maintaining a clear separation of concerns.

1.4.1 1.4.1 Backend Framework: Spring Boot

The core business logic and server-side operations are implemented using Spring Boot.

- **Spring Security with JWT:** Ensures secure, stateless authentication and authorization for users and administrators.
- **Spring Data JPA:** Provides efficient ORM-based interaction with

the PostgreSQL database for managing users, events, and bookings.

- **Embedded Server:** Uses an embedded Tomcat server for simplified deployment and handling HTTP requests.

1.4.2 1.4.2 Frontend Framework: React.js

The user interface is developed using React.js as a Single Page Application (SPA) to provide a smooth and interactive user experience.

- **Component-Based Architecture:** UI elements such as event listings, booking forms, and dashboards are modular and reusable.
- **State Management:** React Hooks like `useState` and `useEffect` are used to manage application state and dynamic updates.
- **API Integration:** Axios is used for seamless communication with backend REST APIs.

1.4.3 1.4.3 Database and Persistence: PostgreSQL

PostgreSQL is used as the relational database system to store and manage application data.

- **Data Storage:** Stores user details, event information, booking records, and transactions.

- **Data Integrity:** Uses constraints and relationships to maintain consistency across tables.
- **Efficient Queries:** Supports optimized queries for fast data retrieval.

1.4.4 1.4.4 Authentication and Security

Security mechanisms are implemented to protect user data and system access.

- **JWT Authentication:** Provides secure and stateless session management.
- **OTP Verification:** Uses email-based OTP verification for account validation.
- **Role-Based Access:** Differentiates access between users and administrators.

1.4.5 1.4.5 Communication and Services

figure[1.4]:The system integrates various services to enhance functionality and user experience.

- **RESTful APIs:** Enable communication between frontend and backend.

BUILDING A SECURE LOGIN FLOW

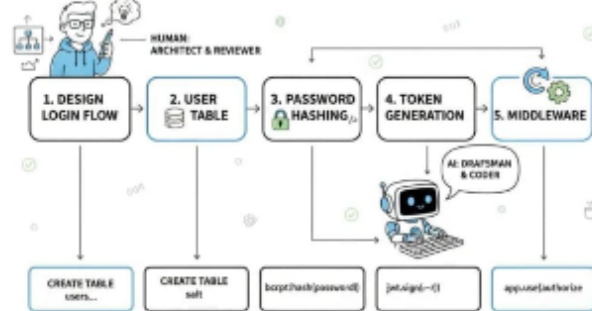


Figure 1.4: Authentication and security

- **SMTP Email Service:** Sends OTPs, booking confirmations, and notifications.
- **Real-Time Updates:** Ensures users receive current booking and event information.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

2.1 Survey of Similar Applications in Market

In recent years, event management and ticket booking systems have evolved significantly, offering advanced features such as online registration, secure payments, real-time tracking, and analytics. Several platforms are widely used in the market, each providing unique functionalities to streamline event organization and ticket sales.

2.1.1 Eventbrite

Eventbrite is one of the most popular online ticket booking platforms used globally for organizing and managing events. It allows users to create event pages, sell tickets, and promote events through integrated marketing tools. The platform supports secure payments, attendee tracking, and automated email confirmations. However, it has

limitations in customization and involves service fees per ticket. :contentReference[oaicite:0]index=0

2.1.2 Zoho Backstage

Zoho Backstage is an all-in-one event management platform that provides tools for event creation, ticket sales, attendee engagement, and analytics. It supports customizable event pages, payment integration, and real-time updates. It is particularly suitable for corporate and large-scale events but may require subscription costs for advanced features. :contentReference[oaicite:1]index=1

2.1.3 Ticket Tailor

Ticket Tailor is a cost-effective ticketing platform designed for small businesses and independent organizers. It offers features such as ticket creation, payment processing, and reporting with minimal service fees. The platform focuses on affordability and simplicity but lacks some advanced analytics and marketing tools compared to enterprise solutions. :contentReference[oaicite:2]index=2

2.1.4 Cvent

Cvent is an enterprise-level event management system that provides comprehensive solutions including registration, ticketing, event mar-

keting, and analytics. It supports large-scale events with advanced customization, reporting, and integration capabilities. However, it is more complex and expensive, making it less suitable for small-scale or internal events. :contentReference[oaicite:3]index=3

2.1.5 TicketSource

TicketSource is a user-friendly ticketing platform widely used for community and small-scale events. It offers features like online ticket sales, event promotion, and real-time reporting. It is easy to use and cost-effective but has limited scalability for large and complex event management scenarios. :contentReference[oaicite:4]index=4

2.1.6 Analysis of Existing Systems

Most existing ticket booking systems provide essential functionalities such as online booking, payment processing, and attendee management. These platforms eliminate manual errors and improve efficiency by centralizing ticket sales and event data. :contentReference[oaicite:5]index=5

However, they also have certain limitations:

- High service charges and subscription costs
- Limited customization for internal institutional use

- Dependency on third-party platforms
- Lack of tailored features for specific organizational needs

2.1.7 Need for Proposed System

Based on the survey, there is a need for a customized ticket booking system that is specifically designed for internal departmental events. The proposed system aims to provide a cost-effective, secure, and scalable solution with features tailored to academic institutions, such as role-based access, wallet integration, and simplified event management.

figure[2.1]:The image presents a comparison of various event management tools based on features, pricing, and usability. It highlights platforms such as EventMobi, Eventbrite, Zoho Backstage, Cvent, Airtable, and Whova. Each tool is evaluated on criteria like best use, ease of use, free features, and pricing structure. Some tools are better suited for large-scale conferences, while others are ideal for small or internal events.

EVENT MANAGEMENT TOOL COMPARISON							
BRAND	PAPERLESS POST	EVENTBOD-IT	EVENTSEN	EVENT	GLUE UP	ADTABLE	HYPERA
BEST FOR	Event teams who value both design and functionality, especially for events of all sizes	Event organizers who value ease of management	Event sales force who do ongoing engagements	Events in the corporate sector, or recurring model, or conferences	Event managers who require more than one	Event time, operational and project management	Conferences and seminars, operational and project management, for all sizes
EASE OF USE	No experience necessary	No experience necessary	No experience necessary	Good for experts	Some experience necessary	No experience necessary	Some experience necessary
FREE FEATURES	Free 1 year trial and 1000 guests, plus 10 free eCards	Free up to 20 guests	Free 100000 guests per event, up to 1000 guests	Free for the design, registration and event guides	50%	Free 1000 guests per event	1000 free trial guests per event
PRICE	\$25-\$7500 event, depending on number of guests and premium features used	Starting at \$2500, plus 20 guests, plus 25% of the value of the event, plus 10% from each ticket sold and a 10% processing fee	Event fees: \$250-\$1000 per month	Customized quote required	Event fees: \$1000 per month	Starting at \$250-\$1000 per month	10% of the total price, plus a \$250 per month fee

Paperless
POST

Figure 2.1: Event manage tool

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Environment Setup

The backend of the system is developed using Spring Boot, a powerful Java-based framework for building scalable and secure web applications. The development environment includes Java Development Kit (JDK 17 or above) and Maven for dependency management and project build automation. The application is developed using an integrated development environment such as IntelliJ IDEA or Eclipse. Required dependencies such as Spring Web, Spring Security, Spring Data JPA, and PostgreSQL Driver are configured to support REST API development, authentication, and database connectivity.

3.1.1 Structure of the Project

The backend follows a layered architecture to maintain a clear separation of concerns and improve maintainability. The project is orga-

nized into multiple packages such as Controller, Service, Repository, and Model. The Controller layer handles incoming HTTP requests and sends responses. The Service layer contains the core business logic of the application. The Repository layer interacts with the PostgreSQL database using JPA for data persistence. The Model layer defines entity classes that represent database tables. Additionally, configuration classes are included for security setup, JWT authentication, and application-level configurations.

3.1.2 Execution Procedure

To execute the backend application, the project dependencies are first downloaded using Maven. The application is then started using the command `mvn spring-boot:run` or by running the main class from the IDE. Once the server starts, the backend runs on a local server (typically on port 8080). The REST APIs can be tested using tools such as Postman or through the frontend application. The backend processes user requests, performs necessary operations, interacts with the database, and returns appropriate responses.

figure[3.1]: This image illustrates a basic web application architecture using the MERN stack. It shows how the client machine interacts with the backend server and database. On the client side, the user accesses the application through a web browser. The browser sends

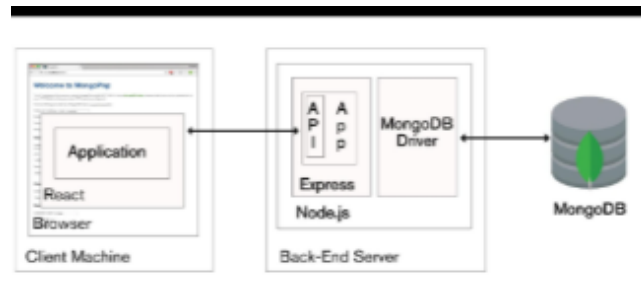


Figure 3.1: Frame work

requests to the server over HTTP. The backend server is built using Node.js and Express.js, which handle routing and business logic.

3.2 Backend Implementation

The backend of the Ticket Booking System is designed using a layered architecture based on Spring Boot. It ensures secure communication, efficient data handling, and smooth interaction with the frontend application.

3.2.1 Environment Setup

The backend environment requires the following tools and technologies:

- **Spring Boot Layer:**

- JDK 17 and Maven for application development and dependency management.

- PostgreSQL for persistent data storage and transaction management.
- Spring Boot Framework for building RESTful APIs and handling server-side logic.

- **Security and Services Layer:**

- Spring Security with JWT for authentication and authorization.
- SMTP Email Service for sending OTP verification and notifications.

3.2.2 Structure of the Project

The backend follows a modular layered architecture to ensure maintainability and scalability:

- **Controller Layer:**

- Handles incoming HTTP requests and sends responses.
- Exposes REST APIs for user authentication, event management, and ticket booking.

- **Service Layer:**

- Contains core business logic such as booking validation, wallet processing, and event handling.
- Acts as an intermediary between Controller and Repository layers.

- **Repository Layer:**

- Manages database operations using Spring Data JPA.
- Performs CRUD operations on user, event, and booking entities.

- **Model Layer:**

- Defines entity classes such as User, Event, Booking, and Wallet.
- Maps Java objects to database tables.

- **Configuration Layer:**

- Handles security configurations, JWT filters, and application properties.

3.2.3 Execution Procedure

The backend application is executed using the following steps:

1. Database Setup:

- Configure PostgreSQL and create the required database.
- Update database credentials in the application.properties file.

2. Run Backend Application:

```
cd backend  
  
mvn clean install  
  
mvn spring-boot:run
```

3. API Testing:

- The backend runs on localhost (default port 8080).
- APIs can be tested using Postman or integrated frontend.
- Verify authentication, event creation, and ticket booking endpoints.

3.3 Database Implementation

The database layer of the system is implemented using PostgreSQL, a powerful and reliable relational database management system. It is used to store and manage all application data such as user details, event information, bookings, and transactions.

3.3.1 Database Configuration

The PostgreSQL database is configured to support efficient data storage and retrieval for the application. A dedicated database is created for the system, and connection details such as database URL, username, and password are specified in the `application.properties` file. Spring Boot uses Spring Data JPA to establish a connection with the database and manage entity persistence. The Hibernate framework is used for automatic table creation and schema updates based on entity classes.

3.3.2 Schema Structure

The database schema is designed using a relational model to ensure data integrity and efficient querying. The schema consists of multiple related tables representing different components of the system. Relationships such as one-to-many and many-to-one are used to link entities like users, events, and bookings. Primary keys uniquely identify each record, while foreign keys maintain relationships between tables. Constraints are applied to enforce data consistency and prevent invalid entries.

3.3.3 Tables Created

The database includes several essential tables to support the system functionality:

- **User Table:** Stores user details such as user ID, name, email, password, and role.
- **Event Table:** Contains information about events including event ID, title, description, date, and available seats.
- **Booking Table:** Maintains records of ticket bookings, linking users with events along with booking status.
- **Wallet Table:** Stores user wallet balance and transaction details for payment processing.
- **Transaction Table:** Records payment transactions and booking confirmations.

3.4 System Technology Stack Overview

The following table provides a comprehensive summary of the frameworks and technologies utilized in the development of the Ticket Booking System, categorized by their architectural role.

Table 3.1: Consolidated System Technology Stack

Component	Technology	Description / Role
Frontend	React.js	Core library for building a dynamic, single-page user interface.
	Axios	Facilitates RESTful API communication between frontend and backend.
	HTML5, CSS3	Used for structuring and styling the user interface.
Backend	Spring Boot	Primary framework for business logic, REST API, and server-side processing.
	Spring Security (JWT)	Provides secure authentication and role-based access control.
	Spring Data JPA	Enables ORM-based database interaction and persistence.
Database	PostgreSQL	Relational database system used for storing users, events, and booking data.
Services	SMTP Email Service	Used for sending OTP verification and booking notifications.
	REST APIs	Enables communication between frontend and backend systems.
Tools	Maven	Dependency management and project build tool.
	Postman	API testing and debugging tool.
	VS Code / IntelliJ	Development environments for frontend and backend.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The architecture of the Ticket Booking System follows a Multi-Tier Architecture, designed to ensure scalability, security, and efficient data handling.

- **Presentation Layer (Frontend):** Built using React.js, this layer provides an interactive user interface for users to register, browse events, and book tickets. It communicates with the backend through RESTful APIs.
- **Application Layer (Backend):** The Spring Boot server acts as the core of the system, handling business logic, user authentication using JWT, event management, and ticket booking operations. It also manages communication between frontend and database.
- **Service Layer (Email & Security Services):** This layer includes

SMTP-based email services for OTP verification and notifications. It also handles role-based access control and secure authentication mechanisms.

- **Data Layer (Database):** PostgreSQL is used as the relational database to store user information, event details, booking records, and transaction data. It ensures data consistency and efficient retrieval.

figure[4.1]: This image represents an Entity-Relationship (ER) diagram for a ticket booking or event management system database.

The Events table stores information about events including event

ID, address, name, organizer ID, and timestamps. The Events_{hasUser} table

to-many relationship between users and events. This means a single user

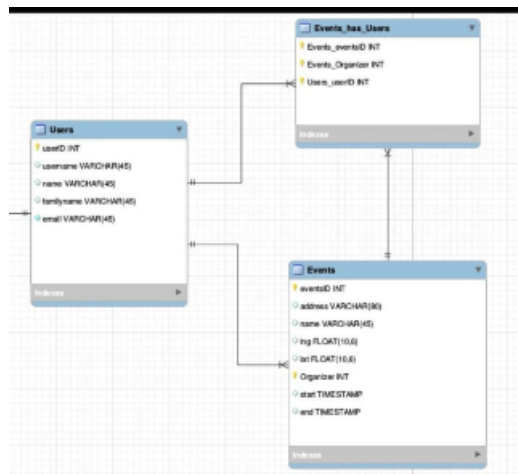


Figure 4.1: data layer

4.2 Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) represents how data flows within the Ticket Booking System from user input to backend processing and database storage.

4.2.1 DFD Level 0: Context Diagram

The Level 0 diagram shows the system as a single process interacting with external entities such as users, database, and email services. Users perform actions like registration, login, and ticket booking, while the system processes and stores the data.

figure[4.1]: This image shows a Level 0 Data Flow Diagram (DFD), also known as a context diagram, for an event management system. It represents the system as a single central process interacting with external entities. The main process, labeled “Event Management System,” handles all core functionalities. On the left side, the user interacts with the system by making bookings and requesting reports.

4.2.2 DFD Level 1: Functional Decomposition

figure[4.2]: The Level 1 diagram divides the system into core processes including user authentication, event management, ticket booking, payment processing, and notification services. These processes

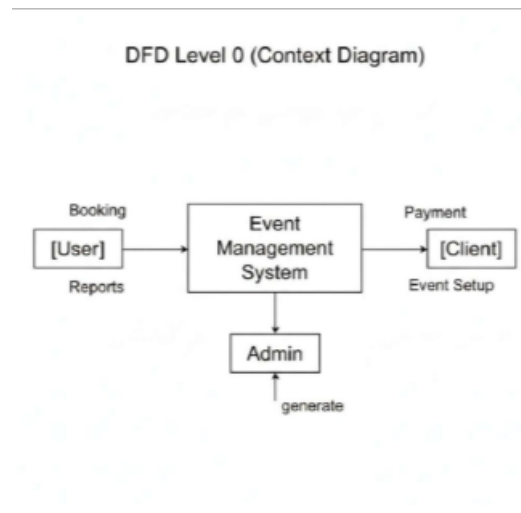


Figure 4.2: DFD Level 0

interact with the database to store and retrieve data.

4.3 Advantages of this Project

- **Automation:** Simplifies event booking and reduces manual work.
- **Security:** Provides secure login using JWT and OTP verification.
- **Real-Time Access:** Enables instant event viewing and ticket booking.

Disadvantages

- **Internet Dependency:** Requires a stable internet connection.
- **Setup Complexity:** Initial configuration requires technical knowledge.
- **Maintenance:** Needs regular updates and monitoring.

figure[4.3]:This diagram represents the Data Flow Diagram (DFD) Level 1 of a ticket booking system, showing how data moves between different components. It illustrates three main processes: Login, Events, and Payment, which handle user interactions. The User initiates actions such as logging in, viewing events, and making payments. The Login process verifies user credentials using the Login database and allows access to the system. The Events process handles event management, including adding, deleting, and updating events, which is managed by an Admin or Manager. The Payment process handles payment transactions, interacting with the Payment database.

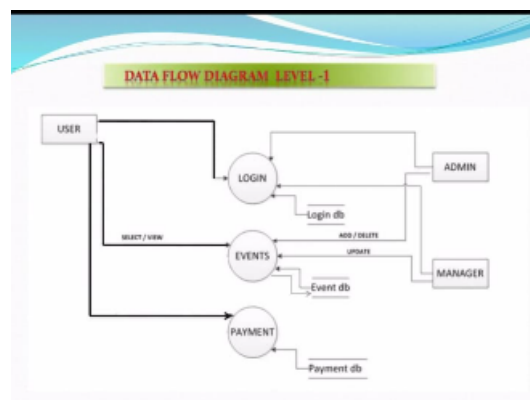


Figure 4.3: DFD Level 1

Chapter 5

RESULTS AND DISCUSSIONS

5.1 System Implementation Overview

The Ticket Booking System is designed as a comprehensive web-based platform that integrates multiple modules for efficient event management and ticket booking.

- **Landing Page Interface:** Provides users with an overview of available events and system features.
- **Authentication Module:** Handles user registration, login, and OTP-based email verification.
- **User Dashboard:** Allows users to view events, manage bookings, and track transaction history.
- **Admin Dashboard:** Enables administrators to create, update, and manage events and monitor bookings.
- **Event Management Module:** Manages event creation, details,

scheduling, and seat availability.

- **Ticket Booking Module:** Allows users to select events, book tickets, and confirm bookings.
- **Wallet & Payment Module:** Handles wallet balance, payment processing, and transaction verification.
- **Notification System:** Sends OTPs, booking confirmations, and updates via email.
- **Database Layer:** Stores user data, event details, bookings, and transaction records using PostgreSQL.

5.2 Module Explanations

5.2.1 User Authentication – Login Page

figure[5.2]:The login interface enables registered users to securely access the system by entering valid credentials. It ensures authentication using proper validation mechanisms to maintain system security.

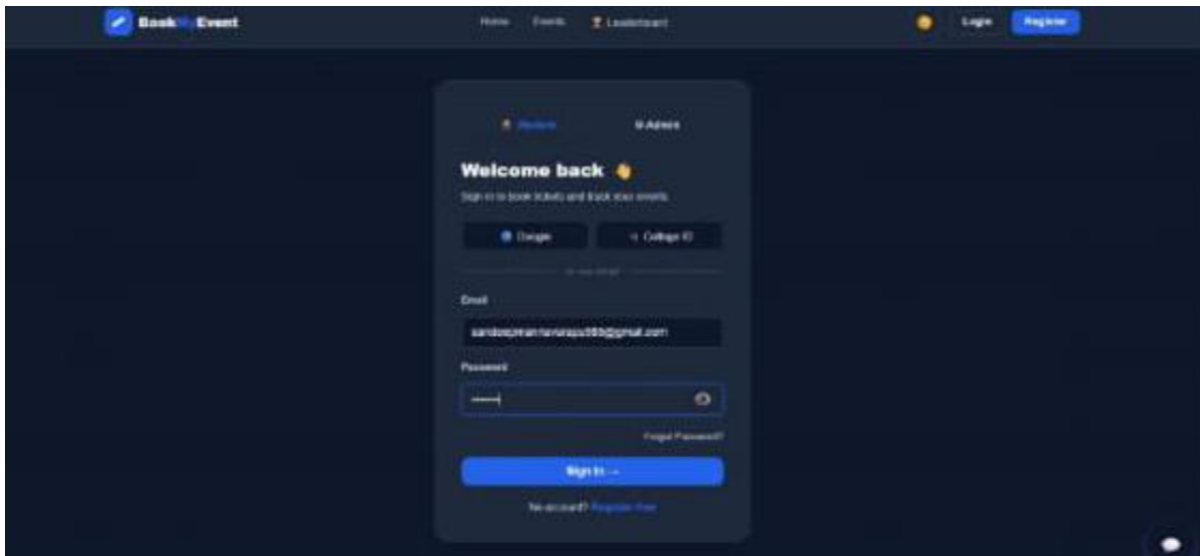


Figure 5.1: login page

5.2.2 User Verification Module

figure[5.2.2]:The verification page provides an additional layer of security by validating user identity, typically through OTP or email verification. This helps prevent unauthorized access.

Figure 5.2: Verification page

5.2.3 Events Module

figure[5.3]:The events page displays available college events in a structured format. Users can browse event details such as title, date, and description, allowing them to choose relevant events.

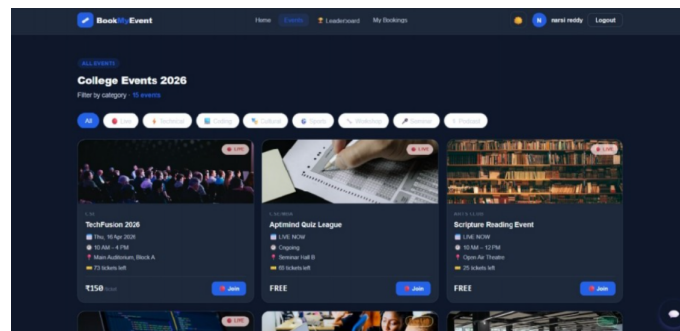


Figure 5.3: Module page

5.2.4 Ticket Booking Module

figure[5.4]:The booking interface allows users to reserve tickets for selected events. It captures necessary booking details and ensures a smooth reservation process.

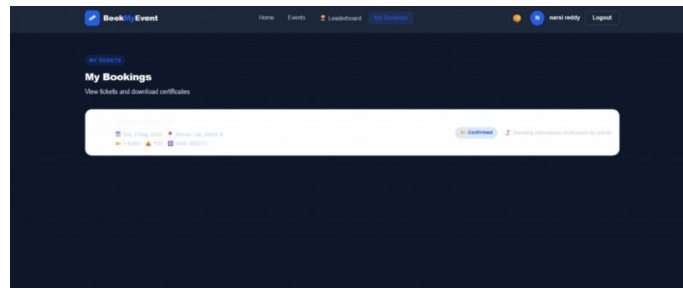


Figure 5.4: Booking page

5.2.5 Wallet Management System

figure[5.5]:The wallet page manages user transactions within the platform. It enables users to view balance, add funds, and make payments for event bookings efficiently.

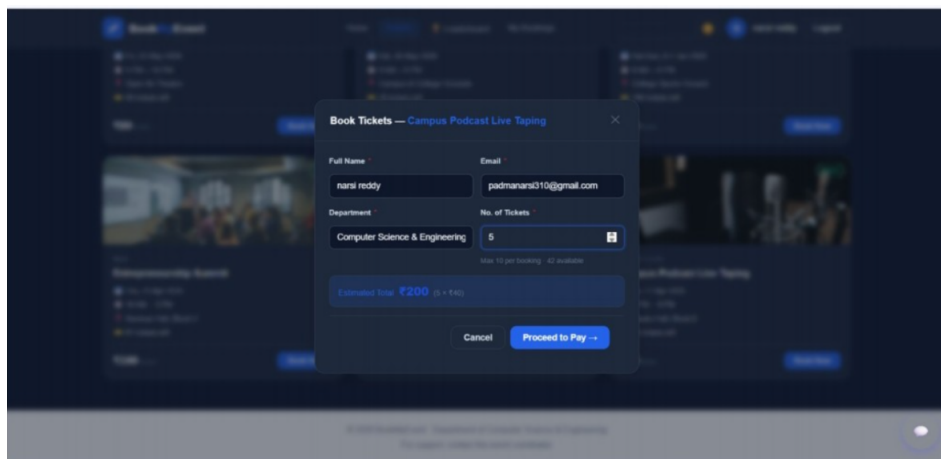


Figure 5.5: Wallet management

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Event Ticket Booking System successfully provides a streamlined and efficient platform for managing college events and reservations. By integrating user authentication, event browsing, ticket booking, and wallet management into a single system, the application ensures a smooth and user-friendly experience.

The project demonstrates that:

1. **Improved Accessibility:** Users can easily explore and book events without manual intervention, reducing time and effort.
2. **Secure Transactions:** The wallet system ensures safe and reliable payment handling within the platform.
3. **Efficient Event Management:** Administrators can manage events

and monitor bookings effectively.

4. **User Convenience:** A centralized system enhances usability and improves the overall booking experience.

In conclusion, the system serves as a reliable and scalable solution for college-level event management, reducing complexity while improving efficiency and accessibility.

6.2 Future Enhancements

While the current system provides a strong foundation, the following enhancements can further improve its functionality:

- **QR Code-Based Entry:** Generate QR codes for tickets to enable fast and secure event check-in.
- **Online Payment Integration:** Integrate payment gateways such as UPI, cards, or net banking for real transactions.
- **Admin Analytics Dashboard:** Provide graphical insights on bookings, revenue, and user activity.
- **Real-Time Notifications:** Implement email or SMS alerts for booking confirmation and event updates.
- **Mobile Application:** Develop a mobile app for better accessibility and user engagement.

- **Refund and Cancellation System:** Allow users to cancel bookings and process refunds efficiently.

Chapter 7

References

[1] LinkedIn: Professional Networking and Global Job Portal.

<https://www.linkedin.com>

[2] Indeed: Global Job Search and Application Aggregator.

<https://www.indeed.com>

[3] React.js Official Documentation.

<https://reactjs.org>

[4] Spring Boot Documentation.

<https://spring.io/projects/spring-boot>

[5] FastAPI Documentation.

<https://fastapi.tiangolo.com>

[6] PostgreSQL Documentation.

<https://www.postgresql.org/docs/>

[7] MDN Web Docs (HTML, CSS, JavaScript).

<https://developer.mozilla.org>

[8] JWT (JSON Web Token) Introduction.

<https://jwt.io>

[9] Razorpay Payment Gateway Documentation.

<https://razorpay.com/docs>

[10] REST API Design Best Practices – Microsoft.

<https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design>

[11] GitHub Documentation.

<https://docs.github.com>

[12] Docker Documentation.

<https://docs.docker.com>

[13] I. Sommerville, *Software Engineering*, Pearson Education.

[14] M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media.

[15] R. C. Martin, *Clean Code*, Prentice Hall.